Модуль NN

Клиент-Сервер

КлиентСервер • NextJS • TypeScript • Компоненты • Валидация • Backend • API • Формы

В предыдущих модулях мы создали REST API (Express + MongoDB/Mongoose или Express + PostgreSQL/Prisma). В этом модуле мы создадим фронтенд на Next.js с TypeScript, который будет взаимодействовать с бэкендом из раздела 7.9 (модуль 7).

Модуль краткий и практический: мы сосредоточимся на минимуме, необходимом для создания клиентской части.

NN.1 Введение в Next.js и TypeScript

Что такое Next.js?

Next.js — это React-фреймворк для создания веб-приложений. Он добавляет к React серверный рендеринг (SSR), маршрутизацию на основе файлов, оптимизацию производительности и многое другое.

Почему Next.js, а не чистый React?

- Маршрутизация из коробки: каждый файл в папке app становится страницей.
- Server Components: компоненты по умолчанию рендерятся на сервере, что улучшает SEO и производительность.
- Встроенная оптимизация: шрифты, изображения, код автоматически оптимизируются.
- API Routes: можно создавать API-эндпоинты прямо в Next.js (но мы будем использовать наш отдельный бэкенд).

Что такое TypeScript?

TypeScript — это надмножество JavaScript с системой типов. Он помогает находить ошибки на этапе написания кода, а не в процессе выполнения кода.

TypeScript проверяет, какие данные используются в программе, как они передаются между частями кода, и соответствует ли всё ожидаемой структуре. Благодаря этому разработчик получает подсказки,

предупреждения и более точное понимание того, как работает приложение.

После проверки TypeScript преобразует код в обычный JavaScript, который уже выполняется в браузере или на сервере.

Быстро о TypeScript за 5 минут

TypeScript позволяет явно указывать тип данных.

Типы переменных:

```
// Базовые типы
const name: string = 'Алексей'; // Строка
const age: number = 25; // Число
const isActive: boolean = true; // Булево


// Массивы
const tags: string[] = ['node', 'react']; // Массив строк
const ids: number[] = [1, 2, 3]; // Массив чисел
```

Интерфейсы — описывают, как должен выглядеть объект:

```
interface Author {
    id: number;
    name: string;
    email: string;
    bio?: string; // знак вопроса ? — это необязательное поле
}


interface Book {
    id: number;
    title: string;
    isbn: string;
    pages?: number;
    published: boolean;
    authorId: number;
    author?: Author; // связанный объект (опционально)
}
```

Типизация функций:

```
function greet(name: string): string {
    return `Привет, ${name}!`;
}


// Стрелочная функция с типами
```

```
const sum = (a: number, b: number): number => a + b;


// Функция без возвращаемого значения
function log(message: string): void {
    console.log(message);
}
```

Типизация React-компонентов:

```
// Props — свойства компонента
interface AuthorCardProps {
    author: Author;
    onDelete: (id: number) => void;
}


function AuthorCard({ author, onDelete }: AuthorCardProps) {
    return (
        <div>
            <h3>{author.name}</h3>
            <button onClick={() =>
onDelete(author.id)}>Удалить</button>
        </div>
    );
}
```

Этого достаточно для нашего модуля. TypeScript будет подсказывать ошибки прямо в редакторе кода.

NN.2 Создание проекта Next.js

На самом деле фронтенд можно делать на любом фреймворке, да и хоть на чистом HTML/CSS/JS, но в данном модуле мы будем использовать именно Next.js

Создадим новый Next.js проект, выполнив команду:

```
npx create-next-app@latest frontend --typescript --tailwind --app --
eslint
cd frontend
```

Флаги:

--typescript: включает TypeScript.

--tailwind: добавляет Tailwind CSS для стилизации.

--app: использует App Router (новый подход к маршрутизации Next.js).

--eslint: добавляет линтер.

На все остальные вопросы можно ответить значениями по умолчанию.

Структура будущего проекта

```
frontend/
├── app/
│   ├── layout.tsx          # Корневой layout (общая обёртка)
│   ├── page.tsx            # Главная страница /
│   ├── globals.css         # Глобальные стили
│   ├── authors/
│   │   └── page.tsx        # Страница /authors
│   └── books/
│       └── page.tsx        # Страница /books
├── components/             # Переиспользуемые компоненты
├── lib/
│   └── api.ts              # Функции для запросов к API
├── types/
│   └── index.ts            # Типы TypeScript
├── next.config.ts
├── tailwind.config.ts
├── tsconfig.json
└── package.json
```

Запуск dev-сервера

```
npm run dev
```

Откройте http://localhost:3000 — вы увидите стартовую страницу Next.js.

Настройка переменных окружения

Бэкенд и фронтенд работают на разных портах. Фронтенд Next.js по умолчанию запускается на порту 3000, а бэкенд мы запустим на порту 4000, чтобы они не конфликтовали. Создайте файл .env.local в корне проекта frontend:

```
NEXT_PUBLIC_API_URL=http://localhost:4000/api
```

Префикс NEXT_PUBLIC_ делает переменную доступной в клиентском коде (в браузере). Без этого префикса переменная будет доступна только в серверных компонентах Next.js

Настройка CORS на бэкенде

Чтобы фронтенд мог делать запросы к бэкенду с другого порта, добавьте CORS в бэкенд. Откройте проект бэкэнда и установите пакет:

```
npm install cors
```

В server.js бэкенда импортируйте пакет cors и подключите:

```
import cors from 'cors';

app.use(cors({
    origin: 'http://localhost:3000', // адрес фронтенда
}));
```

Немного о CORS (Cross-Origin Resource Sharing)

Это механизм безопасности в браузере, который контролирует, может ли один сайт получать данные с другого сайта.

Проще говоря: когда веб-страница делает запрос к серверу на другом домене, браузер сначала проверяет, разрешает ли сервер отдавать данные этому сайту.

Например:

- сайт siteA.com отправляет запрос к api.siteB.com
- браузер смотрит ответ сервера
- если сервер в ответе указал специальный заголовок, разрешающий доступ — данные будут получены
- если такого разрешения нет — браузер блокирует запрос

Это сделано для безопасности, чтобы вредоносные сайты не могли автоматически получать данные с других сервисов от имени пользователя (например, из личного кабинета или банковского сервиса).

Важно понимать, что **CORS работает именно в браузере**.

Если отправить тот же запрос с сервера, через curl или Postman, он обычно выполнится без ограничений.

Подробнее можно почитать в официальной документации:

https://developer.mozilla.org/ru-RU/docs/Web/HTTP/CORS

На этом бэкенд пока оставим и перейдем к фронтенду.

NN.3 Типы и API-клиент на фронтенде

Определение типов

В Next.js приложении создайте файл types/index.ts с типами, соответствующими моделям бэкенда:

```
export interface Author {
    id: number;
    name: string;
    email: string;
    bio?: string;
    books?: Book[];
    createdAt: string;
    updatedAt: string;
}


export interface Book {
    id: number;
    title: string;
    isbn: string;
    pages?: number;
    published: boolean;
    authorId: number;
    author?: Author;
    createdAt: string;
    updatedAt: string;
}
```

Типы должны совпадать с тем, что возвращает ваш API.

API-клиент

Создайте файл lib/api.ts — единое место для всех запросов к бэкенду:

```
const API_URL = process.env.NEXT_PUBLIC_API_URL ||
'http://localhost:4000/api';

async function fetchAPI<T>(path: string, options?: RequestInit):
Promise<T> {
    const res = await fetch(`${API_URL}${path}`, {
        headers: { 'Content-Type': 'application/json' },
        ...options,
    });

    if (!res.ok) {
        const error = await res.json().catch(() => ({}));
        throw new Error(error.error || `Ошибка: ${res.status}`);
    }

    if (res.status === 204) return {} as T;
    return res.json();
}
```

Функция fetchAPI — обёртка над fetch. Она автоматически добавляет заголовок Content-Type, обрабатывает ошибки и парсит JSON.

Добавьте в файл api.ts функции для авторов:

```
import { Author, Book } from '@/types';

// Авторы
export const getAuthors = () =>
    fetchAPI<{ authors: Author[]; total: number; page: number; pages:
number }>('/authors');

export const getAuthor = (id: number) =>
    fetchAPI<Author>(`/authors/${id}`);

export const createAuthor = (data: { name: string; email: string; bio?:
string }) =>
    fetchAPI<Author>('/authors', {
        method: 'POST',
        body: JSON.stringify(data),
    });
```

```
export const updateAuthor = (id: number, data: Partial<Author>) =>
    fetchAPI<Author>(`/authors/${id}`, {
        method: 'PATCH',
        body: JSON.stringify(data),
    });


export const deleteAuthor = (id: number) =>
    fetchAPI<void>(`/authors/${id}`, { method: 'DELETE' });
```

И функции для книг:

```
// Книги
export const getBooks = (params?: string) =>
    fetchAPI<{ books: Book[]; total: number; page: number; pages:
number }>(`/books${params ? `?${params}` : ''}`);


export const getBook = (id: number) =>
    fetchAPI<Book>(`/books/${id}`);


export const createBook = (data: {
    title: string;
    isbn: string;
    pages?: number;
    authorId: number;
}) =>
    fetchAPI<Book>('/books', {
        method: 'POST',
        body: JSON.stringify(data),
    });


export const updateBook = (id: number, data: Partial<Book>) =>
    fetchAPI<Book>(`/books/${id}`, {
        method: 'PATCH',
        body: JSON.stringify(data),
    });


export const deleteBook = (id: number) =>
    fetchAPI<void>(`/books/${id}`, { method: 'DELETE' });

export const searchBooks = (q: string) =>
    fetchAPI<Book[]>(`/books/search?q=${encodeURIComponent(q)}`);
```

Обратите внимание: Partial<Author> означает, что все поля Author становятся необязательными. Это удобно для PATCH-запросов, где мы обновляем только часть полей.

NN.4 Страницы и маршрутизация

App Router в Next.js

В Next.js (App Router) маршрутизация основана на файловой системе. Каждая папка в app/ — это сегмент URL, а файл page.tsx — страница для этого маршрута:

```
app/page.tsx              →  /
app/authors/page.tsx      →  /authors
app/authors/[id]/page.tsx → /authors/1, /authors/2, ...
app/books/page.tsx        →  /books
```

Файл layout.tsx — общая обёртка (навигация, шапка, футер). Он применяется ко всем вложенным страницам.

Server Components и Client Components

В Next.js компоненты бывают двух видов:

- Server Components (по умолчанию): рендерятся на сервере. Могут делать fetch напрямую, но не могут использовать useState, useEffect, обработчики событий (onClick).
- Client Components (с директивой 'use client'): рендерятся в браузере. Могут использовать хуки React и обработчики событий.

Правило: используйте Server Components для отображения данных, Client Components — для интерактивности (формы, кнопки, состояние).

Корневой layout

Отредактируйте app/layout.tsx:

```
import './globals.css';
import Link from 'next/link';
export default function RootLayout({
    children,
}: {
    children: React.ReactNode;
}) {
    return (
        <html lang="ru">
            <body className="min-h-screen bg-gray-50">
```

```
                <nav className="bg-white shadow p-4 mb-6">
                    <div className="max-w-4xl mx-auto flex gap-6">
                        <Link href="/" className="font-bold text-lg">
                            Библиотека
                        </Link>
                        <Link href="/authors" className="text-blue-600
hover:underline">
                            Авторы
                        </Link>
                        <Link href="/books" className="text-blue-600
hover:underline">
                            Книги
                        </Link>
                    </div>
                </nav>
                <main className="max-w-4xl mx-auto px-4">
                    {children}
                </main>
            </body>
        </html>
    );
}
```

Link из next/link — аналог <a>, но с клиентской навигацией (без полной перезагрузки страницы). Tailwind CSS классы стилизуют навигацию.

Главная страница

Отредактируйте app/page.tsx:

```
export default function Home() {
    return (
        <div className="text-center py-20">
            <h1 className="text-4xl font-bold mb-4">Библиотека</h1>
            <p className="text-gray-600 mb-8">
                Управление авторами и книгами
            </p>
        </div>
    );
}
```

NN.5 Список и отображение данных

Страница авторов

Создайте файл app/authors/page.tsx. Эта страница — Client Component, потому что нам нужен useState для хранения данных и useEffect для загрузки:

```
'use client';


import { useEffect, useState } from 'react';
import { Author } from '@/types';
import { getAuthors, deleteAuthor } from '@/lib/api';
import Link from 'next/link';


export default function AuthorsPage() {
    const [authors, setAuthors] = useState<Author[]>([]);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState('');


    const loadAuthors = async () => {
        try {
            setLoading(true);
            const data = await getAuthors();
            setAuthors(data.authors);
        } catch (err) {
            setError(err instanceof Error ? err.message : 'Ошибка
загрузки');
        } finally {
            setLoading(false);
        }
    };


    useEffect(() => {
        loadAuthors();
    }, []);


    const handleDelete = async (id: number) => {
        if (!confirm('Удалить автора?')) return;
        try {
            await deleteAuthor(id);
            setAuthors(authors.filter((a) => a.id !== id));
        } catch (err) {
            alert(err instanceof Error ? err.message : 'Ошибка
удаления');
        }
```

```
    };

    if (loading) return <p>Загрузка...</p>;
    if (error) return <p className="text-red-600">{error}</p>;

    return (
        <div>
            <div className="flex justify-between items-center mb-6">
                <h1 className="text-2xl font-bold">Авторы</h1>
                <Link
                    href="/authors/new"
                    className="bg-blue-600 text-white px-4 py-2 rounded
hover:bg-blue-700"
                >
                    Добавить автора
                </Link>
            </div>

            {authors.length === 0 ? (
                <p className="text-gray-500">Авторов пока нет</p>
            ) : (
                <div className="space-y-4">
                    {authors.map((author) => (
                        <div key={author.id} className="bg-white p-4
rounded shadow">
                            <div className="flex justify-between">
                                <div>
                                    <h2 className="text-lg font-
semibold">{author.name}</h2>
                                    <p className="text-gray-
500">{author.email}</p>
                                    {author.bio && <p className="mt-
1">{author.bio}</p>}
                                    {author.books && (
                                        <p className="text-sm text-
gray-400 mt-1">
                                            Книг: {author.books.length}
                                        </p>
                                    )}
                                </div>
                                <button
                                    onClick={() =>
handleDelete(author.id)}
                                    className="text-red-600 hover:text-
red-800"
                                >
                                    Удалить
                                </button>
```

```
                            </div>
                        </div>
                    ))}
                </div>
            )}
        </div>
    );
}
```

Разбор:

- 'use client' — директива, делающая компонент клиентским.
- useState<Author[]>([]) — состояние с типизацией: массив Author, начальное значение — пустой массив.
- useEffect загружает данные при монтировании компонента.
- key={author.id} — React требует уникальный ключ для элементов списка.
- handleDelete удаляет автора через API и обновляет локальное состояние.

NN.6 Формы и создание данных

Форма создания автора

Создайте файл app/authors/new/page.tsx:

```
'use client';

import { useState } from 'react';
import { useRouter } from 'next/navigation';
import { createAuthor } from '@/lib/api';

export default function NewAuthorPage() {
    const router = useRouter();
    const [name, setName] = useState('');
    const [email, setEmail] = useState('');
    const [bio, setBio] = useState('');
    const [error, setError] = useState('');
    const [submitting, setSubmitting] = useState(false);

    const handleSubmit = async (e: React.FormEvent) => {
```

```
        e.preventDefault();
        setError('');
        setSubmitting(true);

        try {
            await createAuthor({ name, email, bio: bio || undefined });
            router.push('/authors');
        } catch (err) {
            setError(err instanceof Error ? err.message : 'Ошибка');
        } finally {
            setSubmitting(false);
        }
    };

    return (
        <div className="max-w-md mx-auto">
            <h1 className="text-2xl font-bold mb-6">Новый автор</h1>

            {error && (
                <p className="bg-red-100 text-red-700 p-3 rounded mb-
4">{error}</p>
            )}

            <form onSubmit={handleSubmit} className="space-y-4">
                <div>
                    <label className="block text-sm font-medium mb-
1">Имя *</label>
                    <input
                        type="text"
                        value={name}
                        onChange={(e) => setName(e.target.value)}
                        required
                        className="w-full border rounded px-3 py-2"
                    />
                </div>
                <div>
                    <label className="block text-sm font-medium mb-
1">Email *</label>
                    <input
                        type="email"
                        value={email}
                        onChange={(e) => setEmail(e.target.value)}
                        required
                        className="w-full border rounded px-3 py-2"
                    />
                </div>
                <div>
```

```
                    <label className="block text-sm font-medium mb-
1">Биография</label>
                    <textarea
                        value={bio}
                        onChange={(e) => setBio(e.target.value)}
                        rows={3}
                        className="w-full border rounded px-3 py-2"
                    />
                </div>
                <button
                    type="submit"
                    disabled={submitting}
                    className="bg-blue-600 text-white px-6 py-2 rounded
hover:bg-blue-700 disabled:opacity-50"
                >
                    {submitting ? 'Сохранение...' : 'Создать'}
                </button>
            </form>
        </div>
    );
}
```

Разбор:

- useRouter() из next/navigation — для программной навигации (router.push).
- React.FormEvent — тип события формы. e.preventDefault() предотвращает перезагрузку страницы.
- bio: bio || undefined — если bio пустая строка, передаём undefined (поле необязательное).
- disabled={submitting} — блокирует кнопку во время отправки, чтобы предотвратить двойные клики.

Форма создания книги

Создайте файл app/books/new/page.tsx:

```
'use client';


import { useState, useEffect } from 'react';
import { useRouter } from 'next/navigation';
import { createBook, getAuthors } from '@/lib/api';
import { Author } from '@/types';
```

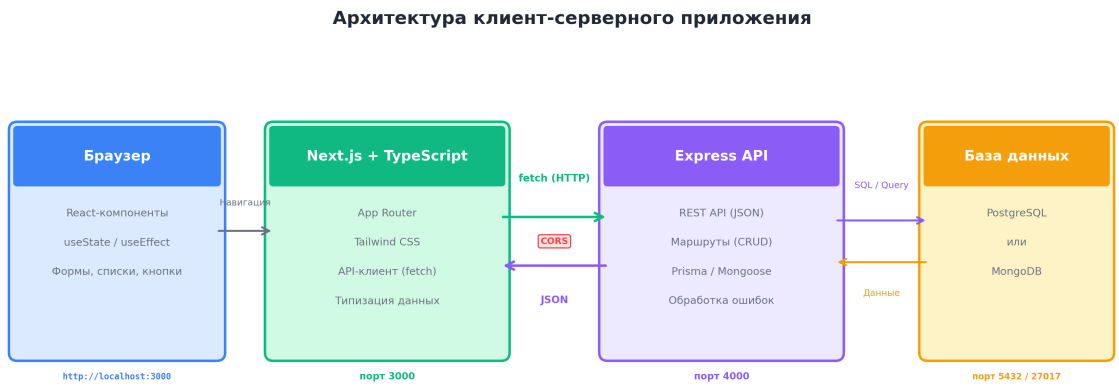
```
export default function NewBookPage() {
    const router = useRouter();
    const [title, setTitle] = useState('');
    const [isbn, setIsbn] = useState('');
    const [pages, setPages] = useState('');
    const [authorId, setAuthorId] = useState('');
    const [authors, setAuthors] = useState<Author[]>([]);
    const [error, setError] = useState('');
    const [submitting, setSubmitting] = useState(false);

    useEffect(() => {
        getAuthors().then((data) => setAuthors(data.authors));
    }, []);

    const handleSubmit = async (e: React.FormEvent) => {
        e.preventDefault();
        setError('');
        setSubmitting(true);

        try {
            await createBook({
                title,
                isbn,
                pages: pages ? Number(pages) : undefined,
                authorId: Number(authorId),
            });
            router.push('/books');
        } catch (err) {
            setError(err instanceof Error ? err.message : 'Ошибка');
        } finally {
            setSubmitting(false);
        }
    };

    return (
        <div className="max-w-md mx-auto">
            <h1 className="text-2xl font-bold mb-6">Новая книга</h1>

            {error && (
                <p className="bg-red-100 text-red-700 p-3 rounded mb-
4">{error}</p>
            )}

            <form onSubmit={handleSubmit} className="space-y-4">
                <div>
```

```
                    <label className="block text-sm font-medium mb-
1">Название *</label>
                    <input
                        type="text"
                        value={title}
                        onChange={(e) => setTitle(e.target.value)}
                        required
                        className="w-full border rounded px-3 py-2"
                    />
                </div>
                <div>
                    <label className="block text-sm font-medium mb-
1">ISBN *</label>
                    <input
                        type="text"
                        value={isbn}
                        onChange={(e) => setIsbn(e.target.value)}
                        required
                        className="w-full border rounded px-3 py-2"
                    />
                </div>
                <div>
                    <label className="block text-sm font-medium mb-
1">Страницы</label>
                    <input
                        type="number"
                        value={pages}
                        onChange={(e) => setPages(e.target.value)}
                        className="w-full border rounded px-3 py-2"
                    />
                </div>
                <div>
                    <label className="block text-sm font-medium mb-
1">Автор *</label>
                    <select
                        value={authorId}
                        onChange={(e) => setAuthorId(e.target.value)}
                        required
                        className="w-full border rounded px-3 py-2"
                    >
                        <option value="">Выберите автора</option>
                        {authors.map((a) => (
                            <option key={a.id} value={a.id}>
                                {a.name}
                            </option>
                        ))}
                    </select>
                </div>
                <button
```

```
                    type="submit"
                    disabled={submitting}
                    className="bg-blue-600 text-white px-6 py-2 rounded
hover:bg-blue-700 disabled:opacity-50"
                >
                    {submitting ? 'Сохранение...' : 'Создать'}
                </button>
            </form>
        </div>
    );
}
```

Обратите внимание на загрузку авторов для выпадающего списка <select>. При создании книги нужно выбрать автора, поэтому мы загружаем список авторов при монтировании компонента.

NN.7 Полное приложение: собираем всё вместе



Прежде чем собрать всё вместе, убедимся, что бэкенд из раздела 7.9 (модуль 7) настроен для работы с фронтендом. Наше приложение состоит из двух независимых серверов:

- Фронтенд (Next.js, порт 3000): отображает интерфейс, отправляет запросы к API.
- Бэкенд (Express, порт 4000): обрабатывает запросы, работает с базой данных, возвращает JSON.

Настройка бэкенда для работы с фронтендом

Мы уже установили пакет cors в разделе 2. Но если вы еще этого не сделали, установите пакет cors в проекте:

```
npm install cors
```

Теперь нужно изменить порт, чтобы он не конфиктовал с фронтендом. Откройте .env бэкенда и измените порт на 4000:

```
PORT=4000
```

Обновите server.js — добавьте импорт и подключение cors:

```
import express from 'express';
import cors from 'cors'; // ← Импорт библиотеки
import authorsRouter from './routes/authors.js';
import booksRouter from './routes/books.js';

const app = express();
const PORT = process.env.PORT || 4000;

// Подключаем библиотеку обработки cors
app.use(cors({
    origin: 'http://localhost:3000', // адрес фронтенда
}));
app.use(express.json());

// Логирование запросов
app.use((req, res, next) => {
    console.log(`${new Date().toISOString()} ${req.method}
${req.url}`);
    next();
});

// Маршруты
app.use('/api/authors', authorsRouter);
app.use('/api/books', booksRouter);

// 404 — маршрут не найден
app.use((req, res) => {
    res.status(404).json({ error: 'Маршрут не найден' });
});
```

```
// Централизованный обработчик ошибок
app.use((err, req, res, next) => {
    console.error(err.stack);
    res.status(err.status || 500).json({
        error: err.message || 'Внутренняя ошибка сервера',
    });
});

app.listen(PORT, () => {
    console.log(`Сервер: http://localhost:${PORT}`);
});
```

Ключевые изменения по сравнению с разделом 7.9:

- Добавлен пакет cors и подключён с origin: 'http://localhost:3000'.
- Порт изменён на 4000, чтобы не конфликтовать с фронтендом Next.js (порт 3000).
- Маршруты и обработчики ошибок остались без изменений.

Теперь опять вернемся к приложению фронтенда и создадим недостающую страницу. Это страница списка книг:

Создайте файл app/books/page.tsx:

```
'use client';

import { useEffect, useState } from 'react';
import { Book } from '@/types';
import { getBooks, deleteBook } from '@/lib/api';
import Link from 'next/link';

export default function BooksPage() {
    const [books, setBooks] = useState<Book[]>([]);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState('');

    const loadBooks = async () => {
        try {
            setLoading(true);
            const data = await getBooks();
            setBooks(data.books);
        } catch (err) {
            setError(err instanceof Error ? err.message : 'Ошибка
загрузки');
        } finally {
```

```
            setLoading(false);
        }
    };

    useEffect(() => {
        loadBooks();
    }, []);

    const handleDelete = async (id: number) => {
        if (!confirm('Удалить книгу?')) return;
        try {
            await deleteBook(id);
            setBooks(books.filter((b) => b.id !== id));
        } catch (err) {
            alert(err instanceof Error ? err.message : 'Ошибка');
        }
    };

    if (loading) return <p>Загрузка...</p>;
    if (error) return <p className="text-red-600">{error}</p>;

    return (
        <div>
            <div className="flex justify-between items-center mb-6">
                <h1 className="text-2xl font-bold">Книги</h1>
                <Link
                    href="/books/new"
                    className="bg-blue-600 text-white px-4 py-2 rounded
hover:bg-blue-700"
                >
                    Добавить книгу
                </Link>
            </div>

            {books.length === 0 ? (
                <p className="text-gray-500">Книг пока нет</p>
            ) : (
                <div className="space-y-4">
                    {books.map((book) => (
                        <div key={book.id} className="bg-white p-4
rounded shadow">
                            <div className="flex justify-between">
                                <div>
                                    <h2 className="text-lg font-
semibold">{book.title}</h2>
                                    <p className="text-gray-500">ISBN:
{book.isbn}</p>
```

```
                                    {book.author && (
                                        <p className="text-sm text-
gray-400">
                                            Автор: {book.author.name}
                                        </p>
                                    )}
                                    {book.pages && (
                                        <p className="text-sm text-
gray-400">
                                            Страниц: {book.pages}
                                        </p>
                                    )}
                                </div>
                                <button
                                    onClick={() =>
handleDelete(book.id)}
                                    className="text-red-600 hover:text-
red-800"
                                >
                                    Удалить
                                </button>
                            </div>
                        </div>
                    ))}
                </div>
            )}
        </div>
    );
}
```

Запуск обоих серверов

Откройте два терминала:

Терминал 1 (бэкенд):

```
cd my-prisma-app
npm run dev
# Сервер: http://localhost:4000
```

Терминал 2 (фронтенд):

```
cd frontend
npm run dev
# Сервер: http://localhost:3000
```

Откройте http://localhost:3000 в браузере. Вы должны увидеть навигацию с ссылками на авторов и книги. Попробуйте создать автора, затем создать книгу с привязкой к автору.

Типичные ошибки и их решение

CORS-ошибка (Access-Control-Allow-Origin):

- Убедитесь, что cors подключён в бэкенде с правильным origin.

Ошибка fetch failed (ECONNREFUSED):

- Бэкенд не запущен или работает на другом порту.

- Проверьте NEXT_PUBLIC_API_URL в .env.local.

TypeScript ошибка Type is not assignable:

- Типы в types/index.ts не совпадают с ответом API.

- Проверьте структуру ответа через curl или Postman.

NN.8 Практика

Задание NN.8.1: Редактирование книг и авторов

Цель: добавить возможность отредактировать книгу и/или автора

Что нужно сделать:

1. Страница /authors/[id]/edit — форма редактирования автора (загрузка текущих данных, PATCH-запрос на редактирование).

2. Страница /books/[id]/edit — форма редактирования книги (загрузка текущих данных, PATCH-запрос на редактирование).

3. Кнопки Редактировать в списках авторов и книг.

Задание NN.8.2 (дополнительное): Поиск и фильтры

Цель: добавить возможность поиска книги по названию и/или автору

Что нужно сделать:

Добавьте на страницу книг:

1. Поле поиска по названию (используя маршрут /books/search?q=).
2. Выпадающий список для фильтрации по автору книги
3. Пагинация (кнопки Назад/Далее).

Заключение

В этом модуле мы изучили:

- Основы TypeScript: типы, интерфейсы, типизация функций и компонентов.
- Создание проекта Next.js с App Router и Tailwind CSS.
- Маршрутизацию на основе файлов, Server и Client Components.
- API-клиент: обёртку над fetch с типизацией.
- Загрузку и отображение данных (useState, useEffect).
- Формы: создание записей, обработка ошибок, состояние загрузки.
- Связку фронтенд + бэкенд: CORS, переменные окружения, два сервера на разных портах.

Теперь вы умеете создавать полноценные клиент-серверные приложения: от базы данных через REST API до интерфейса пользователя.